Who Knew What, When, and Why: A Replayable, Bitemporal Memory Substrate for AI Agents

Marko Vidrih
Creatus.ai
marko@creatus.ai

July 2026

Abstract

AI agents now produce work faster than the people responsible for that work can track it. The memory systems attached to those agents have become the de facto record of decisions, yet none of the deployed architectures can answer the questions any organization eventually asks of a record: who learned this fact, when, from what source, and what belief did it replace. We analyze twelve memory systems across the current design space and find a shared root cause: they treat memory as a mutable cache of extracted facts, so provenance is destroyed at write time and history is destroyed at update time. We propose the Bitemporal Provenance Memory Architecture (BPMA). Its core commitment is a canonical, append-only event record from which every recall structure, working memory, semantic facts, summaries, graphs, and retrieval indexes, is a derived, versioned, replayable projection. Every Memory Event and every extracted Claim carries its principal (human, agent, or pipeline, with model version), bitemporal coordinates (event time and ingestion time), and lineage to its sources. We release Memora, an open-source reference implementation, and AuditEval, a benchmark of five provenance query classes that existing conversational benchmarks do not test. On AuditEval, Memora answers 100% of origin, supersession, attribution, belief-at-time, and contamination queries by construction, while an otherwise identical destructive-update store answers 19.2%, 19.2%, 100%, 38.1%, and 0%. Single-threaded SQLite ingestion sustains about 5,800 events per second with sub-millisecond current-belief lookups at 50,000 events, and full projection rebuilds are bit-identical across replays. Code and benchmark are available at https://github.com/Nifty0x/memora.

1 Introduction

Database engineering answered its accountability questions decades ago. A schema migration log states when a column was added and by whom. A transaction log supports point-in-time recovery. An audit table records which user changed which row. Ask the equivalent questions of the memory layer attached to a modern AI agent, when did the agent learn this fact, from whom, and what did it believe before, and no deployed system returns an answer.

This gap was tolerable when memory meant chat personalization. It is not tolerable now. Agents write production code, negotiate with customers, and operate inside enterprises where decisions made by humans and decisions made by agents mix in a single stream of record. Regulation adds pressure: obligations to explain automated decisions presuppose the ability to reconstruct what a system knew at decision time. The memory layer is becoming the system of record for AI work, and it was not built to be one.

Conventional memory

"What does Acme pay?"

45 EUR

(no source, no author, no history)

BPMA answer card

"What does Acme pay?"

value	45 EUR
written by	`marko` (human)
valid from	2026-07-07
supersedes	40 EUR by `agent.notetaker`
transform	`claim_extract@1.0`
sources	`ev_ded9bd...` (correction note)

Figure 1: The same question answered without and with provenance. The BPMA answer card is the actual output of the Memora walkthrough of Section 6.1: value, author and author kind, validity interval, superseded belief, producing transform, and source Memory Events.

We survey the current generation of agent memory systems in Section 2: context-window managers in the MemGPT line [1], extraction pipelines such as Mem0 [2], temporal knowledge graphs such as Zep [3], associative substrates such as A-MEM [4] and HippoRAG [5], memory operating systems [6], multi-agent and multimodal designs [7, 8], and file-based and verbatim-first local systems. These systems differ in substrate and retrieval strategy, yet they share three failures with one root cause.

First, fidelity loss: extraction pipelines paraphrase experience into facts and discard the original, and controlled ablations show verbatim context outperforming extracted artifacts on long conversations [10]. Second, history loss: update operations overwrite or delete prior state, and consolidation passes rewrite memory in place with no replay path [4]. Third, trust loss: retrieval returns answers with no lineage, so a correct answer and a hallucinated or injected one [14] are indistinguishable to the caller. The root cause is architectural: these systems make the derived view (the extracted fact, the graph edge, the summary) the source of truth, and discard the evidence that produced it.

Databases solved the same problem with event sourcing, immutable logs, and bitemporal modeling [19, 20, 21]. We apply that discipline to agent memory.

Thesis. Agent memory should be organized around a canonical, append-only event record. Everything an agent recalls, working memory, semantic facts, summaries, knowledge graphs, retrieval indexes, is a projection: derived from the record by named, versioned transforms, disposable, and reproducible bit for bit. Provenance and bitemporal validity are structural fields of every record, not optional metadata.

Contributions.

1. A design-space analysis of twelve current memory systems along update discipline, temporal awareness, and provenance depth, and a taxonomy of ten challenges (CH1–CH10) grounded in published failures (Sections 2 and 3).

2. The Bitemporal Provenance Memory Architecture (BPMA): record model, layer structure, and query semantics in which provenance, bitemporality, and replayability are structural rather than bolted on (Sections 4 and 5).

3. Memora, an open-source reference implementation in dependency-free Python over SQLite, covering the ledger, versioned transforms, two projection types, hybrid retrieval with provenance-carrying answers, quarantine, and erasure with receipts (Section 6).

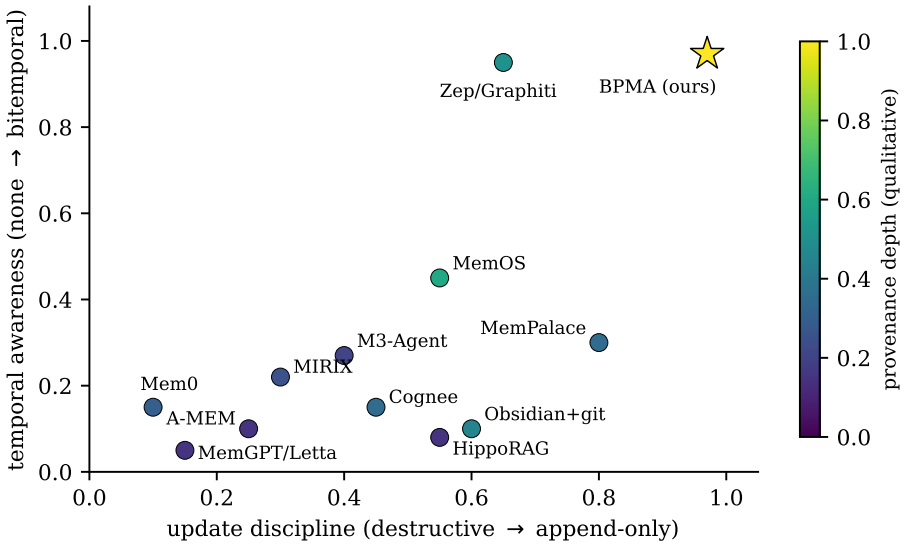

Figure 2: The agent-memory design space. Horizontal: update discipline, from destructive in-place mutation to append-only. Vertical: temporal awareness, from none to full bitemporality (event time and ingestion time with validity intervals). Color: depth of provenance capture (qualitative scoring from Table 1). The upper-right region, append-only with bitemporal semantics and deep provenance, is unoccupied. BPMA targets it.

4. AuditEval, a benchmark of five provenance query classes (origin, supersession, attribution, belief-at-time, contamination) with a deterministic generator and public harness, plus measured results: Memora at 100% across all classes against a destructive-update baseline at 19.2/19.2/100/38.1/0, and systems measurements up to 50,000 events (Section 7).

2 Background and Related Work

We group systems by architectural family. For each we identify the mechanism, its strength, and the specific gap BPMA closes. Figure 2 places the systems in the design space; Table 1 summarizes.

2.1 Context-window management

MemGPT frames the context window as main memory in an operating system and lets the model page information between in-context tiers and external storage through self-editing tool calls [1]; Letta is its production runtime. The strength is a clean control loop. The gap: the agent edits its own memory blocks in place, so the prior contents of an overwritten block are unrecoverable and unattributed.

2.2 Extraction pipelines

Mem0 processes each conversation turn through an extraction phase and an update phase in which an LLM selects ADD, UPDATE, DELETE, or NOOP against the top-k similar stored memories; a graph variant stores entity-relation triplets [2]. This is the dominant production pattern. Two costs follow. Extraction is lossy, and recent controlled ablations report that retrieval over verbatim chunks

outperforms retrieval over extracted artifacts on long conversations [10]. UPDATE and DELETE destroy prior state: the store cannot say what it believed last month or why the belief changed.

2.3 Temporal knowledge graphs

Zep's Graphiti engine maintains episode, semantic-entity, and community subgraphs, with bitemporal bookkeeping on edges: each relation records when it became valid and when it was invalidated, alongside ingestion time [3]. Zep is the closest prior system to BPMA and demonstrates that temporal modeling pays off on LongMemEval-style questions. Three gaps remain: edges do not record which principal (human or agent, and which model) asserted them; graph consolidation is not a versioned, replayable derivation; and the graph is the primary store rather than one projection among several, so non-graph recall structures fall outside its temporal guarantees.

2.4 Associative substrates

A-MEM organizes memory as a Zettelkasten of atomic notes with LLM-suggested links and periodic consolidation passes that rewrite link structure [4]. The published design keeps no audit trail across rewrites: after consolidation there is no record of the previous organization. HippoRAG builds a knowledge graph at write time and retrieves with Personalized PageRank, inspired by hippocampal indexing [5]. Cognee combines vector and graph indexes under LLM extraction. All three center the derived graph as the substrate; none can reconstruct a prior belief state.

2.5 File-based and human-readable memory

A practitioner ecosystem treats plain markdown as agent memory: Obsidian vaults exposed over MCP servers, and instruction files such as `CLAUDE.md` read by coding agents at session start. The strengths are real and underrated: portability, human inspection, and, when the vault lives in git, an accidental append-only history with authorship. The gaps are the absence of schema, temporal semantics, and scoped access. BPMA keeps the plain-text escape hatch (Section 5.7) and supplies the missing discipline.

2.6 Verbatim-first local systems

MemPalace stores conversations verbatim in a local-first store organized by the method of loci, and reported strong LongMemEval retrieval scores at very low context cost; independent analyses disputed the benchmark methodology within days of its viral release. We draw two lessons. Verbatim retention is the correct instinct, consistent with the ablation evidence [10]. And self-reported scores without a public, replayable harness damage trust, which motivates the release discipline of AuditEval (Section 7.1).

2.7 Memory operating systems

MemOS treats memory as a scheduled system resource and packages content with provenance and versioning metadata in a unit called the MemCube [6]. This is the closest prior work on provenance intent. The difference is structural: in MemOS provenance annotates a mutable unit, while in BPMA provenance is the immutable substrate itself; there is no write path that bypasses it, and bitemporal queries are first-class rather than reconstructed from version metadata. MemMachine similarly argues for ground-truth preservation in personalization memory [16].

Table 1: System comparison. Updates: how the primary store changes (in-place mutation vs. append-only). Temporal: none / event-time / bitemporal. Provenance: lineage from stored items to sources. Attribution: principal identity (human vs. agent, model version) on writes. Y = yes, P = partial, – = no.

System	Substrate	Updates	Temporal	Prov.	Attrib.	Multi-ag.	Multi-mod.	Open fmt.
MemGPT/Letta [1]	context blocks	in-place	–	–	–	P	–	P
Mem0 [2]	facts + graph	in-place	P	P	–	P	–	–
A-MEM [4]	linked notes	rewrite	–	–	–	–	–	–
HippoRAG [5]	knowledge graph	add-only	–	–	–	–	–	–
Cognee	vectors + graph	add+reconsol.	P	P	–	–	P	P
Obsidian vault + git	markdown files	append (git)	–	P	P	–	P	Y
MemPalace	verbatim	append	P	P	–	–	–	P
MemOS [6]	MemCube units	versioned	P	P	P	P	P	P
Zep/Graphiti [3]	temporal KG	edge invalid.	Y	P	–	P	–	–
MIRIX [7]	six typed stores	in-place	P	–	–	Y	P	–
M3-Agent [8]	multimodal graph	online update	P	–	–	–	Y	–
BPMA (ours)	ledger + projections	append-only	Y	Y	Y	Y	Y	Y

2.8 Multi-agent and multimodal memory

MIRIX coordinates six typed memory stores (core, episodic, semantic, procedural, resource, knowledge vault) through per-store manager agents and a meta-manager [7]. M3-Agent runs parallel memorization and control processes over video and audio streams into an entity-centric multimodal graph [8]; TeleMem extends long-term multimodal memory to production settings [9]. These systems answer what to store per modality and who coordinates updates. They do not answer accountability: modality-specific stores fragment lineage, and no principal-level attribution crosses store boundaries.

2.9 Consolidation, forgetting, and sleep-time compute

SCM combines working-memory limits, sleep-stage-inspired consolidation, and algorithmic forgetting [11]. Sleep-time compute improves memory representations offline. BPMA adopts scheduled consolidation and makes each pass a named, versioned transform whose outputs are new records, never edits (Section 5.4).

2.10 Governance and security

GateMem benchmarks multi-principal shared-memory governance across utility, access control, and active forgetting [12]; MemArchitect proposes a policy-driven governance layer [13]. Memory-injection attacks insert adversarial content into long-term stores [14]. Provenance is a defense here: in BPMA a write without an attributable principal is quarantined at the door, and a contamination query enumerates everything derived from a compromised source (Section 5.6). A recent survey of persistent memory, state, and governance in always-on agents frames these requirements at field level [15].

2.11 Benchmarks

LoCoMo evaluates very long-term conversational memory over multi-session dialogues [17]; LongMemEval tests information extraction, multi-session and temporal reasoning, knowledge updates, and abstention [18]. Both are conversational, and neither tests provenance: a system that answers correctly while unable to say where the answer came from scores identically to one that can. AuditEval complements them.

3 Problem Analysis: Ten Challenges

Each challenge names a failure observable in current systems, its consequence, and the requirement it imposes. Table 2 maps coverage.

CH1: Lossy extraction. Extract-then-store pipelines paraphrase experience and discard the original. Consequence: unrecoverable detail and measurable retrieval loss versus verbatim context [10]. Requirement: retain verbatim Memory Events; derive, never replace.

CH2: Destructive consolidation. Update operations and consolidation passes mutate the store in place [2, 4]. Consequence: no diff, no rollback, no answer to "what changed." Requirement: consolidation as versioned transforms writing new records over an immutable log.

CH3: Time blindness. Most stores record at best one timestamp. Consequence: knowledge updates corrupt recall; the store cannot distinguish when a fact was true from when the system learned it. Requirement: bitemporality, event time and ingestion time, plus validity intervals on Claims. Zep demonstrates the value of the temporal half [3].

CH4: Missing attribution. No deployed system records the responsible principal on every write. Consequence: human decisions and agent inferences are indistinguishable in the record. Requirement: principal identity (human, agent, pipeline; model and version) as a structural field.

CH5: Unverifiable recall. Retrieval returns strings without lineage. Consequence: correct, stale, and injected answers look alike [14]. Requirement: every answer carries a chain from Claim through transform to source Memory Events.

CH6: Multi-principal governance. Shared memory across agents and people lacks scoped authority [12, 13]. Requirement: scopes and grants on the ledger; agent-to-agent sharing as a scoped grant, not a merge.

CH7: Multimodal fragmentation. Modality-specific stores break cross-modal lineage [8, 7]. Requirement: one record shape with typed payloads; a transcript Claim must point at the audio event it derives from.

CH8: Unaccountable forgetting. Deletion is silent. Consequence: audits cannot distinguish forgetting from tampering; legal erasure conflicts with recordkeeping. Requirement: declarative retention, tombstones, and erasure receipts that destroy payloads while preserving lineage.

CH9: Benchmark validity. Conversational benchmarks do not test provenance [17, 18], and self-reported scores without public harnesses invite the MemPalace failure mode. Requirement: a provenance benchmark with a deterministic generator and released scorer.

CH10: Lock-in. Proprietary stores without an interchange format make memory a captive asset. Requirement: full export to an open plain-text profile.

4 Design Principles

Nine principles follow from the challenges. Together they define BPMA; any implementation satisfying them is a BPMA system.

Table 2: Challenge coverage by family. Y = addressed, P = partial, – = absent.

Challenge	Extraction [2, 4]	Temp. KG [3]	Mem. OS [6]	Files/verb. (§2)	MA/MM [7, 8]	BPMA (ours)
CH1 fidelity	–	P	P	Y	P	Y
CH2 replayable updates	–	P	P	P	–	Y
CH3 bitemporality	–	Y	P	–	–	Y
CH4 attribution	–	–	P	P	–	Y
CH5 verifiable recall	–	P	P	–	–	Y
CH6 governance	–	P	P	–	P	Y
CH7 multimodal lineage	–	–	P	–	P	Y
CH8 accountable forgetting	–	–	P	–	–	Y
CH9 open benchmark	–	P	–	–	–	Y
CH10 portability	–	–	P	Y	–	Y

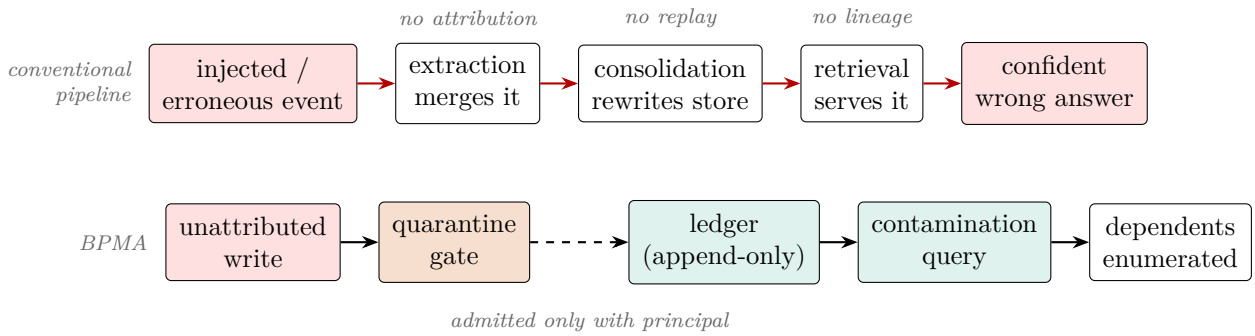

Figure 3: Failure cascade. Top: in a mutable pipeline one bad write propagates through extraction, consolidation, and retrieval, and the trail is destroyed behind it. Bottom: BPMA quarantines unattributed writes at entry, and when a source is later found bad, a contamination query enumerates every Claim derived from it.

P1: Ledger before views. The canonical, append-only event record is the sole source of truth. Anything queryable is derived from it (event sourcing [21, 19]).

P2: Bitemporality everywhere. Every record carries t_{event} (world time) and t_{ingested} (system time); every Claim carries a validity interval $[t_{\text{from}}, t_{\text{to}})$ [20].

P3: Provenance as structure. Principal, principal kind, model reference, source records, and producing transform are non-nullable fields of derived records, not annotations.

P4: Replayable derivation. Transforms are named, versioned, and logged. Rerunning transform version v over the same ledger prefix reproduces its outputs exactly.

P5: Forgetting is policy. Retention rules are declarative; erasure destroys payloads, never lineage, and emits receipts.

P6: Modality-agnostic units. One record shape for text, audio, image, video, tool output, and decisions; payloads are typed, embeddings are per-modality.

P7: Principals and scopes. Every write is attributed to a principal and confined to a scope; cross-scope sharing is an explicit grant.

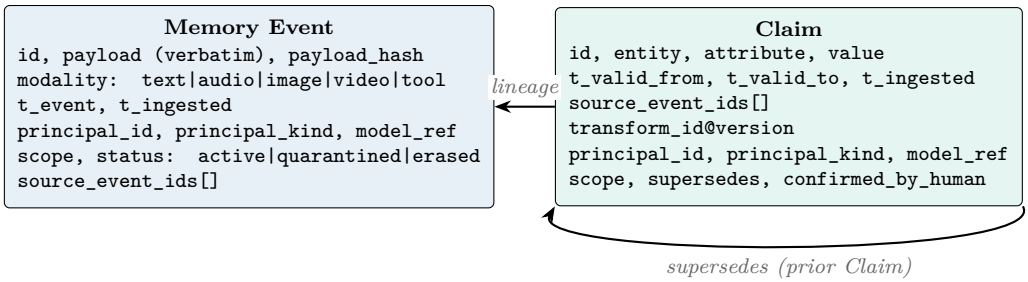

Figure 4: Record anatomy. Provenance and bitemporal fields are structural: no write path can omit them. An erased Memory Event keeps its hash and lineage; only the payload is destroyed.

P8: Plain-text escape hatch. The full ledger exports to markdown plus JSONL; the export is diffable and re-importable.

P9: Retrieval is a planner. Recall routes queries across projections (structured, lexical, vector, graph, temporal) and fuses results, keeping provenance intact end to end.

5 The Bitemporal Provenance Memory Architecture

5.1 Record model

BPMA defines five record-level terms, used consistently throughout.

Memory Event. An immutable entry in the ledger: verbatim payload (any modality), payload hash, t_{event}, t_{ingested}, principal, scope, status, and source-event lineage when the event is itself derived (a transcript derived from audio).

Claim. An extracted assertion: (entity, attribute, value) with validity interval, source Memory Events, producing transform and version, principal, and a supersession link to the Claim it replaced.

Trace. A linked sequence of events and claims tied to one activity (a deal, an incident), materialized by following lineage.

Projection. A derived recall structure (vector index, lexical index, entity graph, summary set) built from the ledger by a transform and disposable at any time.

View. A query-scoped working set assembled by the retrieval planner for one task, with provenance attached.

5.2 Layer structure

Figure 5 shows the six layers. Data flows upward on write and downward on read; the governance plane cuts across all layers.

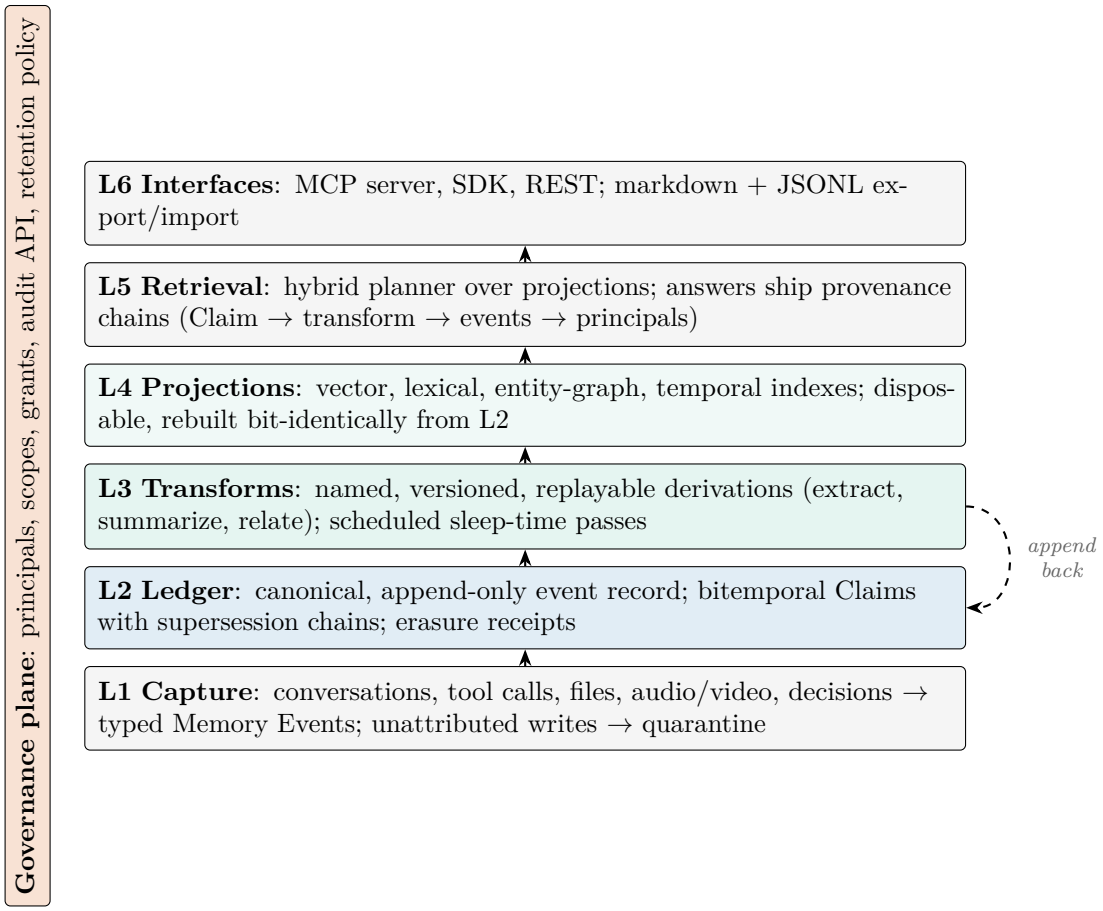

Figure 5: BPMA end to end. The ledger (L2) is the only source of truth. Transforms (L3) read the ledger and append derived records; they never edit. Projections (L4) are disposable indexes. The governance plane spans every layer: no write without a principal, no read outside a scope, no deletion without a receipt.

5.3 Bitemporal semantics

A Claim is true in world time over $[t_{\text{from}}, t_{\text{to}})$ and known to the system from t_{ingested}. Two query families follow. *Belief-at* asks what was true at world time t: the Claim whose validity interval contains t. *As-of* asks what the system would have answered at system time s: restrict to records with $t_{\text{ingested}} \leq s$, then apply belief-at. The distinction matters operationally: a correction ingested today about last month changes belief-at for last month, and leaves as-of answers from yesterday reproducible, which is exactly what an audit requires. Supersession is the write-side dual: asserting a Claim for an (entity, attribute) that has an open Claim closes the open interval at the new t_{from} and links the records (Figure 6).

5.4 Transforms and replayable consolidation

A transform is a pure function from a ledger prefix to derived records, identified by `transform_id@version`. Extraction (events to Claims), summarization (event sets to summary events), and relation mining (Claims to graph edges) are all transforms. Three rules give consolidation without history loss: transforms only append; every output records its inputs and its transform identity; and changing a transform means publishing a new version, whose outputs coexist with the old until a retention policy retires them. Consolidation quality can then improve over time, including with

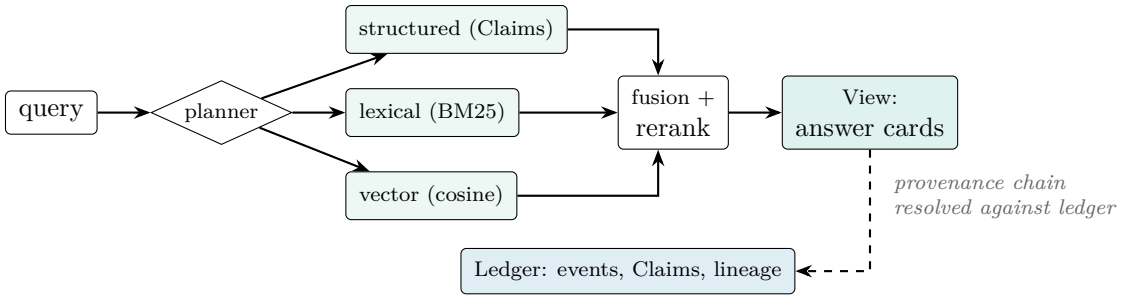

Figure 8: Retrieval as a planner over projections. Provenance survives fusion: each returned item resolves to Claims, transforms, source events, and principals.

better LLMs, while every past belief remains explainable by the transform version that produced it. Figure 7 shows a derivation DAG; the replay property is verified empirically in Section 7.2.

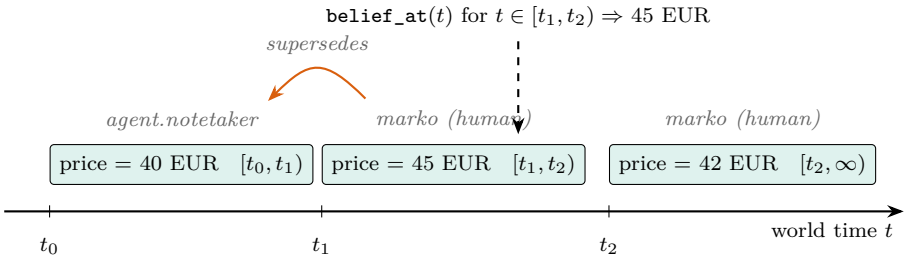

Figure 6: Supersession chain for one (entity, attribute). Nothing is deleted: each value keeps its validity interval, author, and link to what it replaced. `belief_at` resolves any world time to the value then true; as-of queries replay what the system knew at any ingestion time.

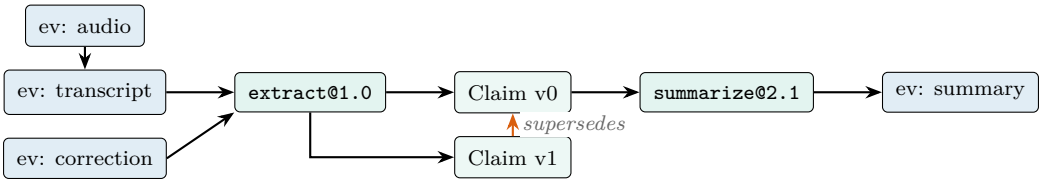

Figure 7: Derivation DAG. Every edge is recorded: derived events carry source lineage, Claims carry source events and transform identity, and supersession links order beliefs. Rerunning `extract@1.0` over the same ledger prefix reproduces Claims v0 and v1 exactly.

5.5 Projections and retrieval

Projections are disposable indexes built by transforms: at minimum a lexical index and a vector index over active events, optionally an entity graph over Claims and a temporal index over intervals. Because projections are derived, index migrations, embedder upgrades, and schema changes are rebuilds, not migrations of truth. The retrieval planner classifies a request (structured lookup, semantic search, temporal reconstruction), routes it to projections, fuses candidates (reciprocal-rank fusion in the reference implementation), and returns Views whose every item links back to Claims and Memory Events. The answer card of Figure 1 is the minimal contract: value, author, validity, predecessor, transform, sources.

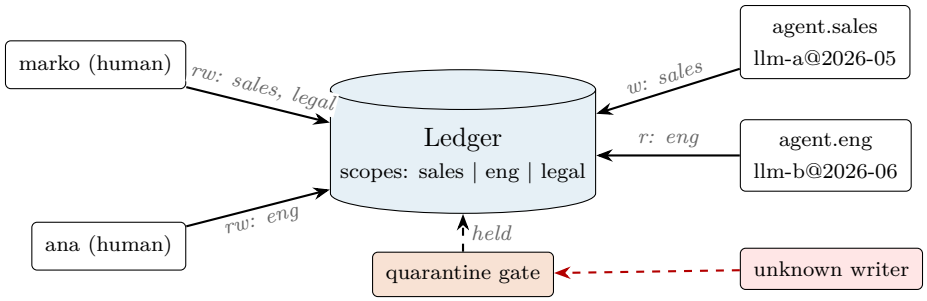

Figure 9: Multi-principal deployment. Humans and agents hold scoped grants on one ledger; every record carries its writer. Unattributed writes stop at the quarantine gate.

5.6 Governance plane

Principals are registered identities: humans, agents (with model reference), and pipelines. Scopes partition the ledger; grants authorize principals to read or write scopes, so agent-to-agent memory sharing is a grant, never a copy, and revocation is instant. Two mechanisms use provenance as a security primitive. Quarantine: a write without an attributable principal is stored with status `quarantined`, excluded from projections until a privileged principal approves it, which counters silent memory injection [14]. Contamination: given a discredited source event, one query enumerates every Claim whose lineage includes it, turning incident response from archaeology into a lookup. The audit API additionally exposes governance sweeps such as "open agent-written Claims never confirmed by a human." Erasure (CH8) destroys payloads, retains hashes and lineage, and emits receipts; the record of what was known remains auditable even when content must go.

5.7 Interfaces and portability

Agents connect over MCP or an SDK; the audit API is part of the same surface, so the questions of Section 7.1 are one call each. The export profile writes one markdown file per Memory Event (frontmatter carries the structural fields) and one JSONL file per table, satisfying P8: the memory of an organization should survive any single vendor, storage engine, or model provider.

6 Memora: A Reference Implementation

Memora implements BPMA in about 700 lines of dependency-free Python over SQLite. The point of the artifact is to prove the architecture executable and measurable, not to win a product comparison: every mechanism claimed in Section 5 exists in code and is exercised by the experiments of Section 7.

The package maps one module per layer: `core.py` (Memory Events, Claims, the ledger, quarantine, erasure receipts, and the transform base class), `projections.py` (a BM25-style lexical projection and a sparse hashing-embedder vector projection, both with canonical state hashes for replay verification), `query.py` (the hybrid retriever with provenance-carrying answer cards, and the audit API), and `baseline.py` (the destructive-update comparator of Section 7.1). Two properties are worth noting. Determinism: the bundled extractor and embedder are deterministic, so replay equality can be checked by hashing; LLM-backed transforms slot into the same interface and are logged by `transform_id@version` either way (Section 8 discusses the determinism boundary). Profiles: the local profile runs SQLite in one file, satisfying the local-first deployment mode that file-based and verbatim-first systems demonstrated demand for; a server profile targets Postgres with pgvector without schema changes.

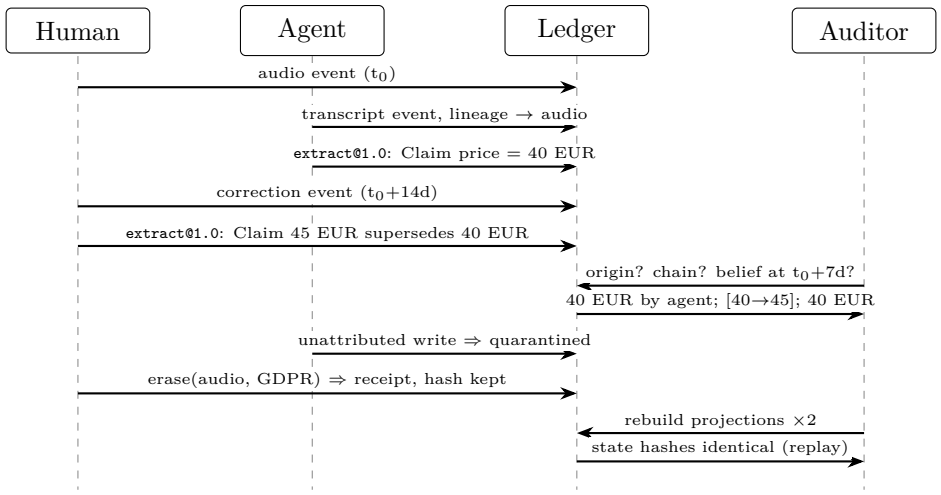

Figure 10: Walkthrough sequence, matching the executable demo. Every arrow into the Ledger appends; nothing is edited in place.

Listing 1: Output of `demo_walkthrough.py` (abridged, verbatim).

```
Q: what is the Acme price?
A: 45 EUR (was 40 EUR), written by marko/human,
   transform transform.claim_extract.rule@1.0, sources ['ev_ded9bdc7b10e']
origin        : {'value': '40 EUR', 'principal_id': 'agent.notetaker',
                 'principal_kind': 'agent', ...}
chain         : [('40 EUR', 'agent.notetaker'), ('45 EUR', 'marko')]
belief mid-way: 40 EUR
contamination : ['cl_bfc69c3273ce']
quarantined   : 1 event(s)
erasure       : er_66b6071d8973 | event payload now: None | hash kept: 12f02caa62da
replay        : lexical True | vector True
ledger stats  : {'events': 4, 'claims': 2, 'erasure_receipts': 1}
```

6.1 End-to-end walkthrough

The repository ships an executable walkthrough whose output is reproduced verbatim in Figure 10 and Listing 1. A meeting recording enters as an audio Memory Event; an agent-derived transcript links to it; an extraction transform asserts the Claim `acme_deal.price = 40 EUR`; two weeks later a human correction supersedes it with `45 EUR`. Retrieval then answers with the full card of Figure 1. The audit API reconstructs origin, the supersession chain, and the mid-interval belief; an unattributed injection attempt lands in quarantine; the audio payload is erased under a GDPR request with a receipt while its hash and lineage survive; and both projections rebuild to bit-identical state hashes.

7 Evaluation

Three questions drive the evaluation. Q-A: does the architecture answer provenance queries that current designs cannot (AuditEval)? Q-B: do the claimed structural properties hold in the running system (functional verification)? Q-C: what does the discipline cost (systems measurements)? All numbers below are produced by the released harness; commands are in Appendix C. Conversational-recall comparisons on LoCoMo [17], LongMemEval [18], and successor benchmarks require external model APIs and are released as a protocol in the repository for the next revision; this paper makes

no unmeasured comparative claims on them.

Table 3: AuditEval accuracy (%), seed 7, 120 entity-attribute pairs, identical inputs and extractor for both systems. Baseline origin and supersession scores equal the share of pairs with a single revision; belief-at-time equals the share of probes in the final interval.

System	Origin	Supersession	Attribution	Belief-at-time	Contamination
Memora (BPMA)	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**
Destructive-update baseline	19.2	19.2	100.0	38.1	0.0

7.1 AuditEval

Design. A deterministic generator (seed 7) creates 40 entities with 3 attributes each and 1 to 4 revisions per attribute (120 entity-attribute pairs), each revision written at a distinct world time by one of three principals: a human and two agents with different model references. The identical revision stream feeds both systems under test through the same extractor, so extraction quality is controlled and the experiment isolates the storage discipline. Five query classes are scored with exact-match answers against generator ground truth: *origin* (first value and first writer), *supersession* (the ordered chain of values), *attribution* (current writer and writer kind), *belief-at-time* (one probe inside every validity interval), and *contamination* (Claims derived from a given revision event).

Comparator. The baseline is the extract-then-update pattern of current pipelines (Section 2): a mutable store keeping the latest value, update time, and last writer per (entity, attribute). It is not a strawman: it receives the same error-free extractions and answers every query as well as its data model permits, including a correct answer for every *attribution* query.

Results. Table 3 and Figure 11. Memora answers 100% in every class; this is by construction, and the run verifies that the construction holds in executable code. The baseline reaches 19.2% on origin and supersession, the fraction of pairs with exactly one revision, where current state and history coincide; 38.1% on belief-at-time, the fraction of probes that happen to fall in the final interval; 0% on contamination, since no lineage exists to traverse; and 100% on attribution, which a last-writer field supports. The pattern, not the magnitudes, is the finding: any store that keeps only current state answers historical provenance queries only when history is trivial, independent of extraction quality, retriever, or model.

7.2 Functional verification

Four structural properties are asserted by the harness on every run. Replay determinism: rebuilding the lexical and vector projections from the ledger twice yields identical canonical state hashes (Listing 1); the projections are therefore disposable in the strong sense. Supersession integrity: for every (entity, attribute), validity intervals partition the timeline with no overlaps and no gaps between consecutive claims. Quarantine: writes without a principal never appear in projections until approved. Erasure: after payload destruction the event's hash, lineage, and receipt remain queryable, and dependent Claims are enumerable through the contamination query.

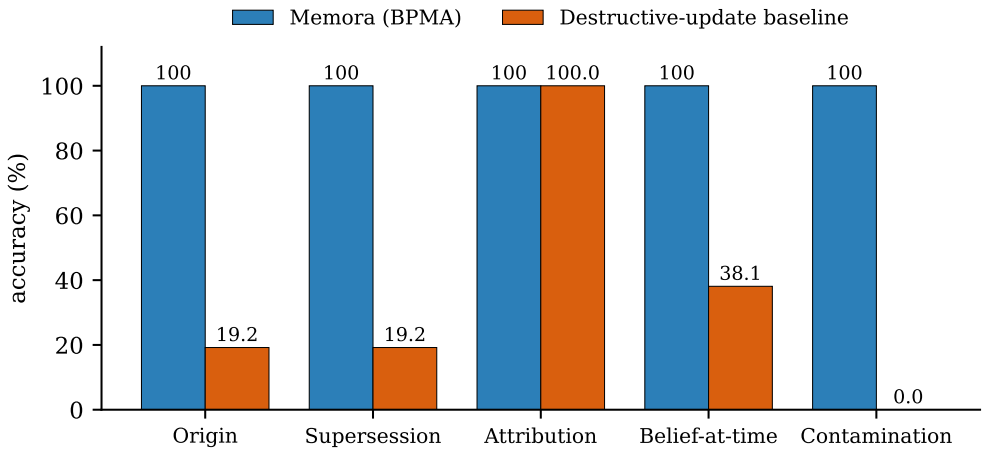

Figure 11: AuditEval results. The baseline fails exactly where history and lineage are required; both systems saw identical inputs.

Table 4: Systems measurements, local profile, single thread, medians. Ingest includes one durable transaction per Memory Event plus extraction. Claim lookup is the current-belief hot path. Search is the dependency-free full-scan hybrid; replay rebuilds both projections from the ledger.

Events	Ingest (ev/s)	Claim lookup p50 (ms)	Search p50 / p95 (ms)	Replay (s)	Ledger (MB)
1,000	7,442	0.009	1.9 / 2.9	0.09	0.75
5,000	7,174	0.009	14.6 / 16.1	0.59	3.61
10,000	6,773	0.010	37.8 / 49.1	1.19	7.17
25,000	5,769	0.015	138.3 / 176.3	3.88	18.60
50,000	5,789	0.010	281.1 / 302.0	5.88	37.86

7.3 Systems measurements

Table 4 and Figure 12 report single-threaded measurements on the local SQLite profile (tmpfs storage, Python 3.10; environment in Appendix C), with one durable transaction per event plus claim extraction. Sustained ingestion stays between 5,800 and 7,400 events per second across ledger sizes. Current-belief lookups, the hot path of an operating agent, hold at 10 to 15 microseconds via the (entity, attribute) index, independent of history depth. Full-scan hybrid search grows linearly to 281 ms at 50,000 events in the dependency-free reference indexes; production profiles replace these with FTS and ANN indexes, and the architecture is indifferent to that substitution because projections are disposable (Section 5.5). Full replay of both projections takes 5.9 s at 50,000 events, bounding the cost of the strongest recovery operation the design supports. Ledger growth is linear at roughly 0.76 KB per event including claims and indexes: the storage price of keeping everything is measured in megabytes per hundred thousand events.

8 Discussion and Limitations

The price of append-only. Storage grows without bound by design. The measured rate (Section 7.3) makes this a policy question rather than a feasibility one, and P5 supplies the mechanism: retention rules compact payloads to receipts while lineage persists. What BPMA refuses is silent loss, not loss.

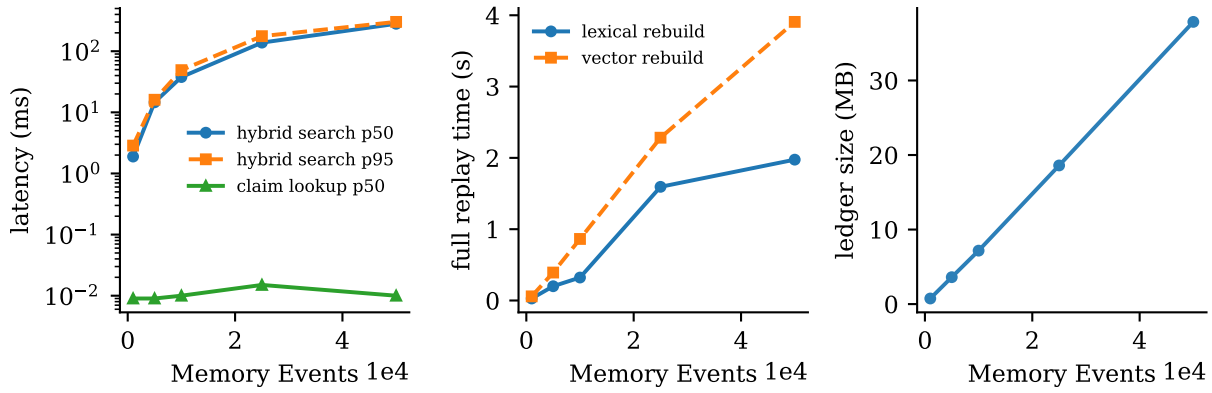

Figure 12: Scaling behavior of the reference implementation. Left: query latency (log scale); the claim-lookup hot path is flat while full-scan search grows linearly and is replaceable by indexed projections. Center: full replay time for both projections. Right: linear ledger growth.

Erasure versus audit. Legal erasure and recordkeeping pull in opposite directions. The receipt mechanism resolves the common case, destroy content, keep the fact that content existed and what depended on it, and a stricter deployment can encrypt payloads per subject and shred keys. Whether receipts satisfy every regulator in every jurisdiction is a legal question this paper does not settle.

The determinism boundary. Replay is bit-exact for deterministic transforms. LLM-backed extraction and summarization are not deterministic across providers and time; BPMA contains this by logging transform identity and version with every output, so replay reproduces *which transform produced what from which inputs* even when rerunning the model would produce different text. Pinned models with fixed decoding narrow the gap; they do not close it.

Scope of the evaluation. AuditEval is self-proposed, synthetic, and adversarially clean: it measures the storage discipline, not extraction quality, retrieval ranking, or end-task success, and the 100% column is a structural consequence, verified rather than earned. The generator, scorer, and seeds are public precisely so the design can be attacked; conversational-recall results against production systems are deferred to the released protocol rather than claimed without measurement. Scoring in Figure 2 and Tables 1 and 2 is a qualitative reading of published designs, not measurements of those systems.

What BPMA does not cover. Parameter memory (weights), activation memory (KV caches), and online learning fall outside the ledger abstraction; MemOS addresses their scheduling [6]. Retrieval quality inherits from projection choice, and nothing prevents pairing BPMA with the strongest published retrievers, graph [3, 5] or otherwise, as projections.

9 Conclusion

The field has treated agent memory as a recall problem: what to store and how to find it again. Deployment makes it an accountability problem: organizations need memory that can say who knew what, when, and why. The Bitemporal Provenance Memory Architecture answers by subordinating every recall structure to a canonical, append-only event record carrying principals, bitemporal

coordinates, and lineage as structural fields. The reference implementation shows the discipline costs microseconds on the hot path and megabytes on disk; AuditEval shows what mutable stores structurally cannot answer, at any extraction quality. Memora and AuditEval are open source under a permissive license. If memory is becoming the system of record for AI work, it should meet the standard the field already holds every other system of record to.

Acknowledgments. The author thanks the open-source agent-memory community, whose published systems and failures made the analysis in Sections 2 and 3 possible.

A Ledger Schema

The complete SQLite schema of the reference implementation (the Postgres profile differs only in types):

```
CREATE TABLE memory_events (
  id TEXT PRIMARY KEY,
  payload TEXT,                -- verbatim; NULL after erasure
  payload_hash TEXT NOT NULL,  -- survives erasure
  modality TEXT NOT NULL,      -- text|audio|image|video|tool|decision
  t_event REAL NOT NULL, t_ingested REAL NOT NULL,
  principal_id TEXT, principal_kind TEXT, model_ref TEXT,
  scope TEXT NOT NULL DEFAULT 'default',
  status TEXT NOT NULL DEFAULT 'active', -- active|quarantined|erased
  source_event_ids TEXT NOT NULL DEFAULT '[]');
CREATE TABLE claims (
  id TEXT PRIMARY KEY,
  entity TEXT NOT NULL, attribute TEXT NOT NULL, value TEXT NOT NULL,
  t_valid_from REAL NOT NULL, t_valid_to REAL, t_ingested REAL NOT NULL,
  source_event_ids TEXT NOT NULL,
  transform_id TEXT NOT NULL, transform_version TEXT NOT NULL,
  principal_id TEXT NOT NULL, principal_kind TEXT NOT NULL, model_ref TEXT,
  scope TEXT NOT NULL DEFAULT 'default',
  supersedes TEXT, confirmed_by_human INTEGER NOT NULL DEFAULT 0);
CREATE TABLE erasure_receipts (
  id TEXT PRIMARY KEY, event_id TEXT NOT NULL, reason TEXT NOT NULL,
  requested_by TEXT NOT NULL, t_erased REAL NOT NULL,
  payload_hash TEXT NOT NULL);
```

B AuditEval Specification

Generator: seed 7; 40 entities $\times$ 3 attributes; revisions per attribute drawn uniformly from $\{1, 2, 3, 4\}$; inter-revision gaps uniform in $[10, 1000]$ seconds of world time; writers drawn uniformly from {one human, two agents with distinct model references}. Each revision is rendered as a structured statement and ingested by both systems through the identical extractor. Scoring is exact match: origin requires first value and first principal; supersession requires the full ordered value chain; attribution requires current principal id and kind; belief-at-time draws one probe uniformly inside every validity interval (open intervals probed within 10^4 s past the last revision); contamination requires exactly the set of Claims derived from the probed event. Reported accuracy is per-class over all probes. The generator, scorer, and both system adapters total under 200 lines and ship in the repository; new systems implement a five-method audit interface to participate.

C Reproducibility

All results in this paper were produced with:

```
python3 -m memora.auditeval          # Table 3 / Figure 10
python3 memora/demo_walkthrough.py   # Listing 1 / Figure 9
python3 memora/bench.py              # Table 4 / Figure 11
python3 memora/plots.py              # Figures 2, 10, 11
```

Environment: CPython 3.10.12, SQLite 3.37.2, single thread, Linux (Ubuntu 22.04) container, database on tmpfs; matplotlib 3.10 for figures. The implementation is dependency-free at runtime. Determinism: seeds are fixed in code; AuditEval and both projections are bit-reproducible (Section 7.2). Measured latencies are medians over 50 search and 200 lookup queries per size; absolute numbers vary with hardware, the reported shape (flat hot path, linear scan, linear growth) does not.

References

[1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[2] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025.

[3] P. Rasmussen et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956, 2025.

[4] W. Xu et al. A-Mem: Agentic Memory for LLM Agents. arXiv:2502.12110; NeurIPS 2025.

[5] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831; NeurIPS 2024.

[6] Z. Li et al. MemOS: A Memory OS for AI System. arXiv:2507.03724, 2025.

[7] Y. Wang and X. Chen. MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv:2507.07957, 2025.

[8] Seeing, Listening, Remembering, and Reasoning: A Multimodal Agent with Long-Term Memory. arXiv:2508.09736, 2025.

[9] TeleMem: Building Long-Term and Multimodal Memory for Agentic AI. arXiv:2601.06037, 2026.

[10] Verbatim Chunks Beat Extracted Artifacts: A Controlled Ablation of Memory Representations for Long LLM Conversations. arXiv:2601.00821, 2026.

[11] SCM: Sleep-Consolidated Memory with Algorithmic Forgetting for Large Language Models. arXiv:2604.20943, 2026.

[12] GateMem: Benchmarking Memory Governance in Multi-Principal Shared-Memory Agents. arXiv:2606.18829, 2026.

[13] MemArchitect: A Policy Driven Memory Governance Layer. arXiv:2603.18330, 2026.

[14] ER-MIA: Black-Box Adversarial Memory Injection Attacks on Long-Term Memory-Augmented Large Language Models. arXiv:2602.15344, 2026.

[15] Always-On Agents: A Survey of Persistent Memory, State, and Governance in LLM Agents. arXiv:2606.30306, 2026.

[16] MemMachine: A Ground-Truth-Preserving Memory System for Personalized AI Agents. arXiv:2604.04853, 2026.

[17] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753, 2024.

[18] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024.

[19] P. Helland. Immutability Changes Everything. CIDR, 2015.

[20] R. T. Snodgrass. Developing Time-Oriented Database Applications in SQL. Morgan Kaufmann, 1999.

[21] M. Fowler. Event Sourcing. martinfowler.com, 2005.